



People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research



University Of Science And Technology Houari Boumediene

Faculty of Computer Science
Department of Computer Science

Master 1 IL Multimedia Module

Mini Project Report

MPEG-4-like Video Compression Codec

Prepared by:

- Kaddour Abdelhadi Said – Matricule: 212131075664
- Issad Abdelmalek – Matricule: 222231371006

Group: 01

Academic Year: 2025–2026

Contents

1	Introduction	2
2	Pipeline Description	2
2.1	Pre-processing	2
2.2	Intra-frame Coding (I-frames)	3
2.3	Inter-frame Coding (P-frames)	4
2.4	Entropy Coding	5
2.5	Decoding and Reconstruction	6
2.6	Evaluation and Visualisation	7
3	Design Choices and Justification	7
4	Experimental Analysis	8
4.1	Effect of the Quantisation Factor f_q	8
4.2	Effect of the GOP Size G	10
4.3	I-frame vs. P-frame Compression Efficiency	11
5	Conclusion	11

1 Introduction

Video compression is a fundamental component of modern multimedia systems. Raw video data is extremely large due to the high spatial resolution of frames and the temporal continuity between them. Efficient compression techniques are therefore essential to reduce storage requirements and enable transmission over bandwidth-limited networks.

The objective of this project is to design and implement a simplified MPEG-4-like video compression pipeline in Python. The system encodes a sequence of image frames into a compact binary representation and supports full reconstruction through decoding.

The approach is based on the core principles used in real codecs:

- Exploitation of spatial redundancy via transform coding,
- Exploitation of temporal redundancy via motion estimation,
- Controlled loss through quantisation,
- Final lossless compression using entropy coding.

2 Pipeline Description

2.1 Pre-processing

Each input frame is converted from BGR to YCbCr colour space. This transformation separates luminance (Y) from chrominance (Cb, Cr), enabling perceptually efficient compression.

Since the human visual system is more sensitive to brightness variations than colour differences, chrominance channels can be compressed more aggressively.

A 4:2:0 chroma subsampling scheme is applied:

- Y is preserved at full resolution,
- Cb and Cr are downsampled by a factor of 2 in both horizontal and vertical directions.

This reduces chroma data to one quarter of its original size, significantly lowering the overall bitrate.



Figure 1: Input frame sequence.

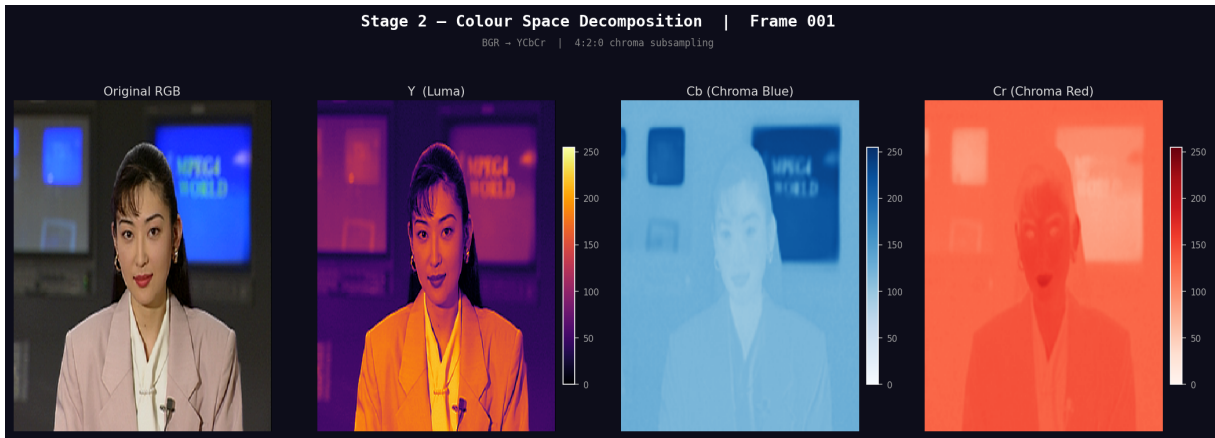


Figure 2: YCbCr decomposition of frame 001.

2.2 Intra-frame Coding (I-frames)

Intra-frame coding processes frames independently and serves as a reference for subsequent frames.

Each channel is divided into 8×8 blocks. For each block:

1. Level shifting is applied:

$$[0, 255] \rightarrow [-128, 127]$$
2. A 2D Discrete Cosine Transform (DCT) is computed,
3. Coefficients are quantised.

The quantisation matrix is defined as:

$$Q(i, j) = 1 + (1 + i + j) \times fq$$

This formulation increases quantisation strength for higher-frequency coefficients, effectively removing fine details that contribute less to perceived quality.

For chroma channels, a stronger quantisation matrix is applied:

$$Q_{chroma} = 2 \times Q_{luma}$$

This is justified by the properties of the human visual system: the eye is significantly less sensitive to fine detail and high-frequency variations in colour than in brightness. As a result, chroma channels can tolerate heavier quantisation with minimal perceptual degradation, allowing greater compression with little visible quality loss.

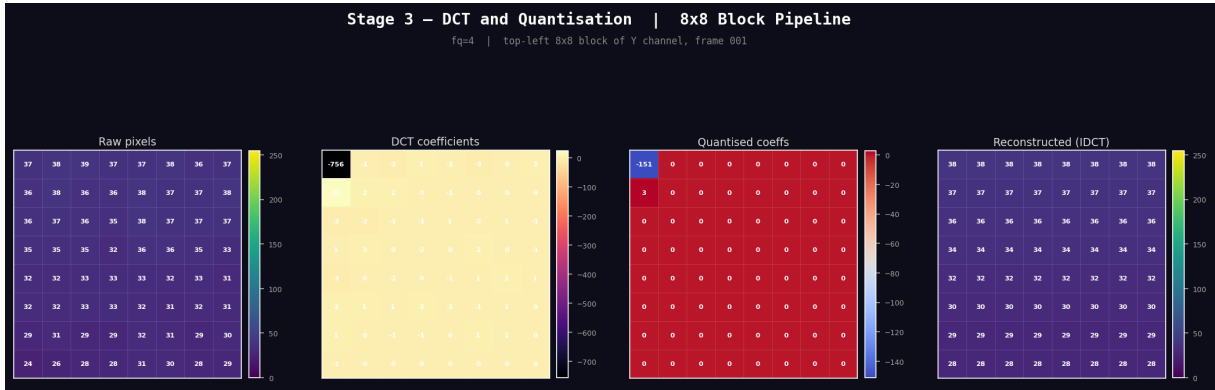


Figure 3: DCT pipeline.

2.3 Inter-frame Coding (P-frames)

Inter-frame coding exploits temporal redundancy between consecutive frames.

A Group of Pictures (GOP) structure is used:

$$I \rightarrow P \rightarrow P \rightarrow \dots \rightarrow I$$

where every G -th frame is encoded independently (I-frame), and others are predicted (P-frames).

Motion Estimation: Each frame is divided into 16×16 macroblocks. For each block, a Full Search Block Matching algorithm is applied within a search window of size $\pm W$.

The best match minimises the Mean Squared Error:

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - y_i)^2$$

where x_i and y_i are the pixel values at position i in the current block and the candidate reference block, respectively, and N is the total number of pixels in the block.

Motion Compensation: The predicted frame is constructed by copying blocks from the reference frame using motion vectors. For chroma channels, motion vectors are scaled due to reduced resolution.

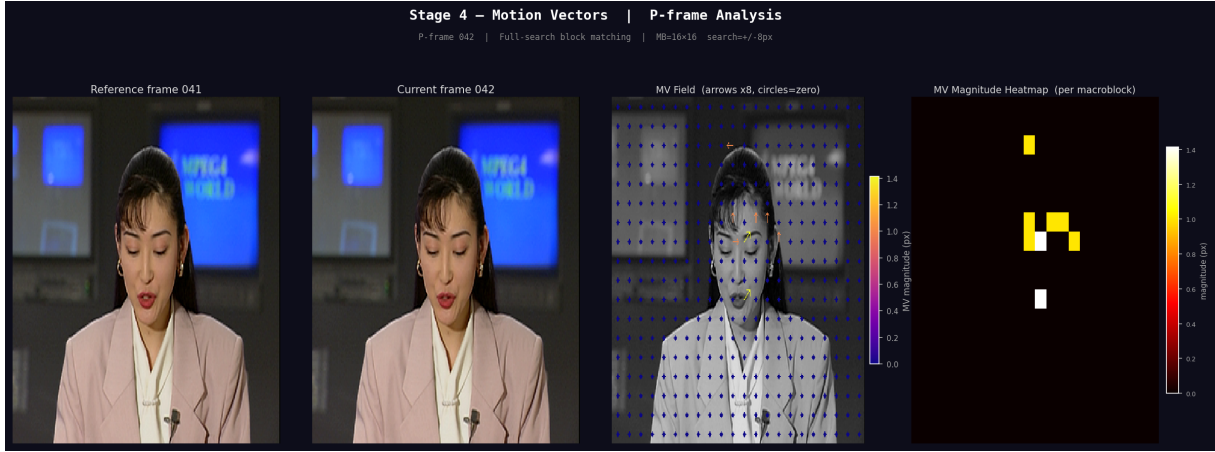


Figure 4: Motion vector field.

Residual Coding:

$$Residual = Current\ Frame - Predicted\ Frame$$

Residuals are encoded using the same DCT and quantisation process as intra coding. Reconstruction is performed as:

$$Reconstructed\ Frame = Prediction + Residual$$



Figure 5: Residual analysis.

2.4 Entropy Coding

After quantisation, data still contains statistical redundancy.

Frames are:

- Serialized using `pickle`,
- Compressed using `zlib` at maximum compression level.

The binary format is structured as:

- Total number of frames (4 bytes),

- For each frame:
 - Compressed size (4 bytes),
 - Compressed data blob.

2.5 Decoding and Reconstruction

The decoding stage is the inverse of the encoding pipeline. It reads the binary bitstream and reconstructs each frame sequentially.

Entropy Decoding: Each compressed frame blob is decompressed using `zlib` and deserialised using `pickle`, recovering the original frame dictionary containing quantised coefficients, motion vectors, and frame metadata.

I-frame Decoding: For each I-frame, decoding proceeds as follows:

1. Dequantisation is applied by multiplying the stored quantised coefficients by the quantisation matrix Q ,
2. The Inverse DCT (IDCT) is computed for each 8×8 block,
3. Level unshifting is applied:

$$[-128, 127] \rightarrow [0, 255]$$

4. The process is repeated independently for the Y, Cb, and Cr channels.

P-frame Decoding: For each P-frame, decoding requires the previously reconstructed reference frame:

1. Motion vectors are read from the bitstream and applied to the *reconstructed* reference frame (not the original) to produce the motion-compensated prediction,
2. The residual is decoded through the same dequantisation and IDCT pipeline as I-frames,
3. The reconstructed frame is obtained by summing prediction and decoded residual:

$$\textit{Reconstructed Frame} = \textit{Prediction} + \textit{Decoded Residual}$$

It is essential that the decoder uses the *reconstructed* reference frame rather than the original. This closed-loop design ensures that the encoder and decoder maintain identical reference states, preventing error drift across successive P-frames.

Reference Frame Buffer: The decoder maintains a reference frame buffer that is updated after each decoded frame. Each newly reconstructed frame immediately replaces the previous reference, so that subsequent P-frames are predicted from the correct reconstructed state.

Chroma Upsampling and Colour Conversion: After decoding, the subsampled Cb and Cr channels are upsampled back to the full luma resolution using bilinear interpolation. The three channels are then merged and converted from YCbCr back to BGR colour space to produce the final reconstructed frame.

2.6 Evaluation and Visualisation

Quality Metric: Compression efficiency is evaluated using the compression ratio:

$$\text{Compression Ratio} = \frac{\text{Original Size}}{\text{Compressed Size}}$$

The system provides:

- Per-frame compression statistics,
- Global compression ratio,
- Distribution of I-frames and P-frames.

Pipeline Visualisation: A dedicated visualisation module illustrates each stage of the encoding pipeline through five figures:

- **Frame sequence (Fig. 1):** sampled input frames annotated with their type (I or P), giving an overview of the GOP structure.
- **Colour space decomposition (Fig. 2):** the Y, Cb, and Cr channels of a single frame displayed separately, making the effect of 4:2:0 subsampling explicit.
- **DCT and quantisation (Fig. 3):** an 8×8 luma block traced through the full transform pipeline — raw pixels, DCT coefficients, quantised coefficients, and the IDCT-reconstructed block.
- **Motion vectors (Fig. 4):** per-macroblock displacement arrows overlaid on the most active P-frame, with a magnitude heatmap identifying the most dynamic regions.
- **Residuals and reconstruction (Fig. 5):** signed and absolute residual maps alongside the motion-compensated prediction and the final reconstructed frame.

3 Design Choices and Justification

YCbCr and 4:2:0 Subsampling: Separating luminance from chrominance allows aggressive compression of colour information with minimal perceptual impact. The 4:2:0 scheme significantly reduces data size while preserving visual quality.

8×8 DCT Blocks: This block size provides strong energy compaction while maintaining reasonable computational complexity, making it a standard choice in image and video compression.

Parametric Quantisation Matrix: The use of the parameter fq enables flexible control over compression strength, allowing systematic tuning between bitrate and visual quality.

Full Search Motion Estimation: Although computationally expensive, this method guarantees the optimal block match within the search window, resulting in more accurate predictions and smaller residuals.

16 × 16 Macroblocks: This size offers a balance between motion estimation accuracy and computational efficiency.

Closed-loop Prediction: Using reconstructed frames as references prevents error accumulation (drift) and ensures consistency between encoder and decoder.

Pickle + Zlib: This combination provides a simple and effective lossless compression stage without adding unnecessary implementation complexity. `pickle` was chosen for its convenience in serialising arbitrary Python data structures, such as NumPy arrays and dictionaries, without requiring a custom binary format. However, it is worth noting that `pickle` is Python-specific and not portable across languages or platforms, which would be a limitation in a production codec context.

4 Experimental Analysis

All experiments were conducted on a sequence of 60 frames at approximately 128 KB each (uncompressed), giving a total raw size of 7704.0 KB. Unless otherwise stated, the fixed defaults are $\text{GOP} = 8$ and $S = 8$.

4.1 Effect of the Quantisation Factor fq

The quantisation factor fq directly scales the quantisation matrix $Q(i, j) = 1 + (1 + i + j) \times fq$, controlling how aggressively DCT coefficients are discarded. A larger fq retains fewer details, producing a smaller bitstream at the cost of visible distortion. To isolate this effect, GOP and search window were fixed at 8 while fq was varied over $\{1, 2, 4, 8, 16\}$.

fq	Total compressed size (KB)	Global compression ratio	I-frame ratio (avg)
1	877.9	8.78×	≈4.26×
2	585.7	13.15×	≈5.96×
4	383.3	20.10×	≈8.22×
8	239.7	32.14×	≈11.54×
16	163.3	47.19×	≈16.44×

Table 1: Effect of fq on compression ($\text{GOP}=8$, $S=8$).

The compression ratio grows steeply and non-linearly with fq : doubling fq roughly doubles the ratio at low values but the gains accelerate as more and more high-frequency coefficients are forced to zero, enabling much shorter entropy-coded representations. At

$fq = 1$ the codec achieves a modest $8.78\times$ ratio; at $fq = 16$ it reaches $47.19\times$, more than five times higher, at the expense of heavy quantisation noise.

Figure 6 plots this evolution across all tested values of fq .

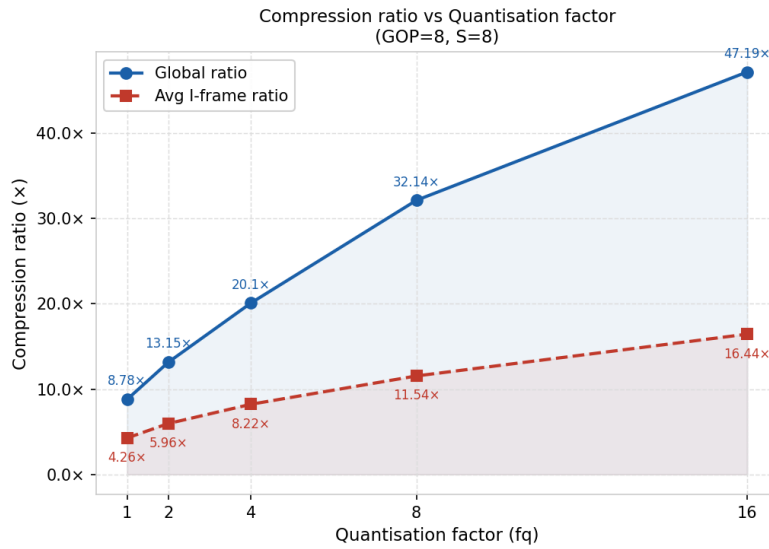


Figure 6: Compression ratio vs quantisation factor fq (GOP=8, $S=8$).

For I-frames the effect is especially dramatic. At $fq = 16$, the average I-frame is compressed to less than 8 KB from an original ~ 128 KB (ratio $\approx 16\times$), while at $fq = 1$ the same frames occupy around 30 KB (ratio $\approx 4.3\times$). P-frames benefit similarly: individual P-frame ratios range from $\sim 10\times$ at $fq = 1$ to values exceeding $90\times$ at $fq = 16$ for the most static macroblocks.

Visually, the transition from $fq = 2$ to $fq = 16$ is clearly perceptible. At $fq = 2$ reconstructed frames retain fine texture and sharp edges. At $fq = 16$, the heavy quantisation forces most non-DC coefficients to zero, producing the characteristic 8×8 block grid (blocking artifacts) and a loss of edge sharpness. Smooth regions of the image are best preserved because their energy is already concentrated in the DC and lowest-frequency AC terms; detailed or textured areas suffer the most degradation at high fq .



Figure 7: Reconstructed frame at $fq = 2$.



Figure 8: Reconstructed frame at $fq = 16$.

4.2 Effect of the GOP Size G

The Group of Pictures size G determines how frequently I-frames are inserted. Every G -th frame is encoded intra (I), and the intervening frames are encoded as P-frames using motion-compensated prediction. A smaller G provides better random access and error resilience (more I-frames), while a larger G exploits more temporal redundancy and therefore achieves better compression. The experiment fixes $fq = 4$ and $S = 8$ and varies G over $\{1, 2, 4, 8, 16\}$.

GOP	Total size (KB)	Compression ratio	I-frames	P-frames
1	934.9	8.24×	60 / 60 (100%)	0 / 60 (0%)
2	633.4	12.16×	30 / 60 (50%)	30 / 60 (50%)
4	465.5	16.55×	15 / 60 (25%)	45 / 60 (75%)
8	383.3	20.10×	8 / 60 (13.3%)	52 / 60 (86.7%)
16	335.3	22.97×	4 / 60 (6.7%)	56 / 60 (93.3%)

Table 2: Effect of GOP size on compression ($fq=4$, $S=8$).

When $G = 1$ every frame is coded independently as an I-frame, and no inter-frame prediction is exploited. The result is a ratio of 8.24×, equivalent to stand-alone image compression. As G increases, the proportion of P-frames grows and the global ratio improves substantially: from 8.24× at $G = 1$ to 20.10× at $G = 8$. Beyond $G = 8$ the gains become progressively smaller (22.97× at $G = 16$), because P-frames that are many steps away from their nearest I-frame accumulate more prediction error, making their residuals larger and harder to compress. This diminishing-return behaviour reflects the fundamental trade-off in GOP selection: longer GOPs improve compression but reduce random-access granularity and increase the cost of error propagation.

Figure 9 illustrates this diminishing-return curve clearly.

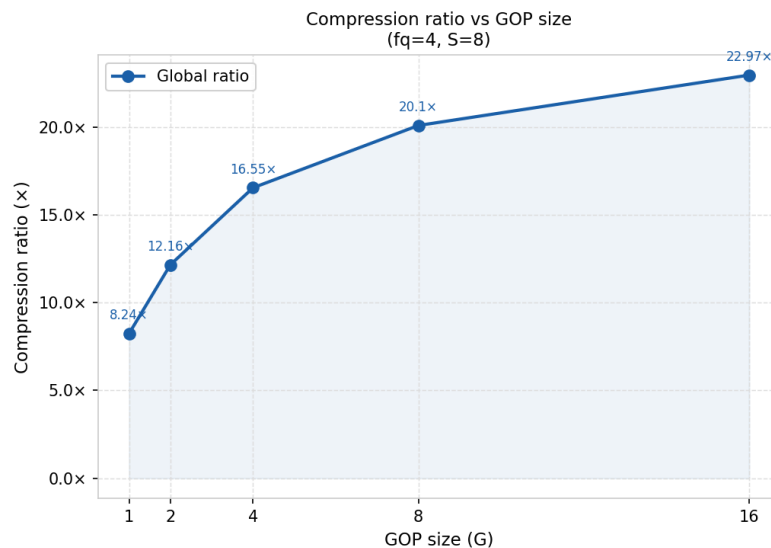


Figure 9: Compression ratio vs GOP size ($fq=4$, $S=8$).

4.3 I-frame vs. P-frame Compression Efficiency

A consistent pattern visible across all experiments is that P-frames are substantially smaller than I-frames. Table 3 illustrates this for the $fq = 4$, $G = 8$, $S = 8$ configuration (the baseline).

Frame	Type	Original (KB)	Compressed (KB)	Ratio
frame_001	I	128.3	15.6	8.24×
frame_002	P	128.4	4.7	27.59×
frame_003	P	128.4	4.1	31.16×
frame_004	P	128.2	3.8	33.75×
frame_005	P	128.3	4.0	32.18×
frame_009	I	128.4	15.6	8.25×
frame_010	P	128.4	4.9	26.37×

Table 3: I-frame vs P-frame sizes ($fq=4$, $GOP=8$).

I-frames consistently occupy around 15–16 KB (ratio $\approx 8\times$) regardless of their position in the sequence. P-frames, by contrast, typically compress to 4–7 KB (ratios of $20\times$ – $34\times$), more than three times smaller. The reason is that motion compensation removes the bulk of the inter-frame signal: the residual — the difference between the motion-compensated prediction and the actual frame — has much lower energy than the full frame and therefore contains far fewer significant DCT coefficients after quantisation. The entropy coder then has a much sparser signal to represent, yielding a much shorter bitstream. Frames that follow a scene change or contain significant global motion (e.g. frame_018 at 6.6 KB) produce larger residuals and therefore a smaller per-frame ratio, but they still outperform I-frames. This consistent advantage of P-frames over I-frames confirms that exploiting temporal redundancy is the dominant source of compression gain in natural video sequences.

5 Conclusion

This project implements a complete video compression pipeline inspired by MPEG-4 principles. By combining transform coding, motion estimation, quantisation, and entropy coding, the system effectively reduces both spatial and temporal redundancy.

The experimental analysis demonstrates three key results. First, the quantisation factor fq is the single most powerful compression lever: increasing fq from 1 to 16 improves the global ratio from $8.78\times$ to $47.19\times$, at the cost of visible blocking artifacts at extreme values. Second, the GOP size has a significant but diminishing impact: moving from all-intra coding ($G = 1$, ratio $8.24\times$) to $G = 8$ ($20.10\times$) nearly triples efficiency, while further increases yield smaller gains. Third, P-frames are consistently and substantially smaller than I-frames because motion-compensated residuals carry far less energy than full intra-coded frames.

The modular design enables detailed analysis and experimentation. The results confirm that significant compression gains can be achieved while maintaining acceptable visual

quality, and that the interplay between fq and GOP size offers a flexible space of rate–quality operating points suited to different application requirements.